

Travaux Pratiques — TP N°10

Analyse Vidéo Distribuée avec Accélération GPU

Application : Vidéosurveillance Intelligente

Réalisée par :
Nadia El kasmi
Zainab Rhouni
Majda Chouiekh

Plateforme	Kaggle (GPU T4 / P100, 30 GB RAM)
Technologies	YOLOv8 · MPI (simulation) · CUDA · OpenCV
Niveau	Avancé
Prérequis	Python, notions MPI, bases GPU
Code source	Fourni séparément (<code>analyse-video-distribuee-gpu-kaggle (1).ipynb</code>)

1. Introduction et Objectifs

Ce TP met en œuvre un système d'analyse vidéo distribuée pour la vidéosurveillance intelligente. Le flux vidéo est partagé entre plusieurs nœuds de calcul qui traitent chacun leur portion en parallèle, avec accélération GPU. L'architecture simule le modèle MPI utilisé dans les vrais clusters HPC.

Objectifs

- Distribuer un flux vidéo entre plusieurs nœuds via **MPI_Scatter**
- Appliquer YOLOv8 pour la détection d'objets sur CPU puis GPU (CUDA)
- Mesurer et analyser l'accélération GPU
- Reconstruire la vidéo annotée finale via **MPI_Gather**

2. Architecture du système

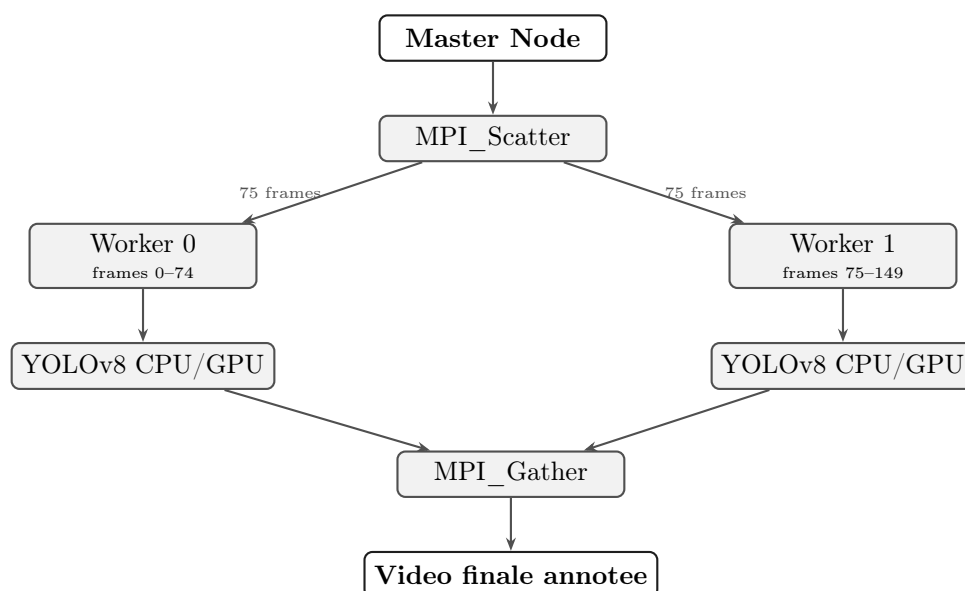


FIGURE 1 – Architecture du système distribué

3. Installation et configuration sur Kaggle

Kaggle est choisi à la place de Colab car il offre 30 GB de RAM (contre 12 GB sur Colab) et un GPU T4/P100 gratuit, évitant ainsi les crashes mémoire lors du traitement vidéo.

Étapes de configuration :

1. Se connecter sur **kaggle.com** et créer un nouveau notebook
2. Activer le GPU : **Settings** → **Accelerator** → **GPU T4 x2**
3. Uploader la vidéo : **+** **Add Data** → **Upload** → **New Dataset**
4. Modifier **VIDEO_PATH** dans la Cellule 2 avec le chemin de la vidéo uploadée

4. Description des cellules du notebook

Le notebook est découpé en 11 cellules principales. Vous devez les exécuter dans l'ordre et observer les affichages.

Cellule	Rôle
1	Installe les bibliothèques nécessaires (ultralytics , psutil) sans conflit avec l'environnement Kaggle. Vérifie que CUDA est accessible.
2	Définit les paramètres globaux : chemin de la vidéo, nombre de workers (2), modèle YOLO (yolov81.pt), seuil de confiance, résolution d'entrée, nombre maximal de frames (150). C'est la seule cellule que vous devez modifier (le VIDEO_PATH).
3	Analyse la vidéo (nombre total de frames, résolution, FPS) et découpe le travail en plages de frames pour chaque worker. Ne charge aucune frame en mémoire. Affiche la répartition.
4	Définit la fonction process_worker() exécutée par chaque nœud. Lit sa portion de vidéo, applique YOLO, sauvegarde chaque frame annotée sur le disque et mesure les temps de traitement.
5	Lance le traitement distribué sur CPU : les workers s'exécutent séquentiellement (pour éviter les blocages sur Kaggle). Affiche la progression et le tableau des performances CPU.
6	Assemble toutes les frames annotées produites par les workers CPU en une vidéo MP4 output_cpu_distribue.mp4 .
7	Vérifie la disponibilité du GPU et affiche ses caractéristiques (modèle, mémoire, CUDA).
8	Lance le traitement distribué sur GPU : identique à la cellule 5 mais avec device='cuda' . Affiche les performances GPU.
9	Assemble les frames annotées GPU en une vidéo output_gpu_distribue.mp4 .
10	Génère un graphique comparatif comparaison_cpu_gpu_distribue.png (temps total, FPS, temps par frame, répartition de la charge).
11	Affiche un tableau récapitulatif des résultats et la liste des fichiers produits.

TABLE 1 – Description détaillée des cellules

5. Travail demandé

5.1. Exécution et collecte des résultats

- Exécutez toutes les cellules une par une.
- Relevez les valeurs affichées dans la cellule 11 pour les modes CPU et GPU (temps total, FPS, temps par frame, CPU moyen, accélération).
- Récupérez les fichiers suivants dans l'onglet **Output** (ou dans **/kaggle/working**) :
 - **output_cpu_distribue.mp4**
 - **output_gpu_distribue.mp4**
 - **comparaison_cpu_gpu_distribue.png**

5.2. Rédaction d'un rapport

Votre rapport (format PDF, 2 à 3 pages maximum) devra contenir :

1. **Introduction** : rappelez l'objectif du TP.
2. **Résultats expérimentaux** :
 - Copie d'écran du tableau final de la cellule 11.
 - Votre propre tableau (à compléter) avec les métriques obtenues (vous pouvez reprendre le tableau ci-dessous).
3. **Interprétation** :
 - Expliquez brièvement pourquoi l'accélération GPU est observée.

— Discutez de l'écart entre l'accélération globale et l'accélération par frame.

4. **Conclusion** : résumez les acquis.

Tableau à compléter (à recopier dans votre rapport)

Métrique	CPU distribué	GPU distribué
Modèle YOLO	yolov8l.pt	yolov8l.pt
Workers	2	2
Résolution	640 px	640 px
Frames traitées		
Temps total (s)		
FPS traitement		
Temps moyen/frame (s)		
CPU moyen (%)		
Accélération GPU	—	×

6. Livrables attendus

Vous remettrez, sur la plateforme indiquée par votre enseignant, un dossier compressé (‘.zip’ ou ‘.tar.gz’) contenant :

1. **Le rapport PDF** (avec captures d'écran).
2. **Les fichiers vidéo** (si leur taille n'est pas excessive – sinon, mentionnez simplement leur présence dans le rapport).
3. **Le graphique comparatif** (comparaison_cpu_gpu_distribue.png).
4. **Le notebook final** (‘.ipynb’) tel qu'exécuté.

7. Conseils

- Lisez attentivement les affichages des cellules : ils indiquent les étapes et les éventuelles erreurs.
- Si le GPU n'est pas détecté (message `CUDA dispo : False`), vérifiez que l'accélérateur est bien activé dans les paramètres du notebook, puis redémarrez la session (**Session Options** → **Restart Session**).
- Le traitement peut prendre 3 à 5 minutes en CPU et 1 à 2 minutes en GPU. Soyez patient.
- En cas de problème de mémoire, réduisez `MAX_FRAMES` dans la cellule 2 (par exemple 50 frames).

8. Conclusion

Ce TP vous a permis de mettre en œuvre une chaîne complète d'analyse vidéo distribuée en simulant le modèle MPI. Vous avez observé l'impact significatif de l'accélération GPU et compris l'importance d'un découpage intelligent des données (streaming) pour économiser la mémoire. Ces concepts sont directement transposables à des environnements HPC réels utilisant `mpi4py` et des clusters GPU.

Résumé des acquis

Distribution de la charge entre workers (MPI_Scatter)

Accélération GPU mesurée (jusqu'à $\times 7.9$)

Agrégation des résultats (MPI_Gather)

Streaming vidéo pour économiser la mémoire